

14-Python List & List Methods Lecture

Introduction to Lists

- Lists are Python's collection (iterable) type, used to store multiple pieces of data (e.g., employee details) in a single variable.
- Useful for grouping related information instead of multiple variables.
- Lists are enclosed in square brackets and can contain zero or more elements.

List Storage and Memory

- Lists store references to their elements, each of which has its own memory address.
- Elements can be of any type (string, int, float, another list, etc.).
- Indexing: Lists support both positive (from 0) and negative (from -1) indices, similar to strings.

List Creation Methods

- **By literal:** Placing elements inside square brackets (e.g., `[10, 'abc', 20.5]`).
- **Using list() constructor:**
Converts an iterable (string, tuple, range, dictionary keys) into a list. Non-iterables (like int) raise an error.
- **From user input:** The `input()` function gives a string; use `eval(input())` to convert to a list.

- **List comprehension:** Concise syntax to build lists using `[expression for item in iterable if condition]`, supports operations and multiple conditions (with ternary operators).
- **Split function:** Converts a string into a list based on delimiters.

Properties of Lists

- **Mutable:**
Lists can be modified after creation (supports item assignment—changes happen in the same list object except for elements replaced).
- **Sequence:**
Ordered collection supporting indexing, slicing (with start:stop:step), concatenation, repetition, and membership testing.
- **Heterogeneous:** Can store elements of different types, including other lists (nested lists).
- **Duplicates allowed:** Lists can have repeated values (unlike sets).
- **Supports nested lists:** Lists can contain lists as elements. Access via chained indices (e.g., `lst[2][1][0]`).

List Operations

- **Indexing and Slicing:** Access elements or sub-lists using `[start:stop:step]`. Negative steps reverse lists; start and stop must be in proper order.
- **Concatenation:** Use `+` to join lists; both operands must be lists.
- **Repetition:** Use `*` with a list and integer to repeat contents.
- **Membership:** `element in list` returns True if found.

List Methods

Counting and Presence

- `count(x)`: Returns number of times `x` appears in the list.
- `index(x[, start[, stop]])`: Returns index of first occurrence of `x`, optional range. Raises error if not found.

Modifying Contents

- `append(x)`: Adds element `x` at the end; only one element per call, can append any type.
- `insert(index, x)`: Inserts `x` at given `index`, shifting others right—no replacements.
- `extend(iterable)`
: Adds all elements from another iterable (like a list) individually at the end. Contrast:
`append([1, 2])` adds `[1, 2]` as one element; `extend([1, 2])` adds `1` and `2` separately.

Removing Elements

- `pop([index])`: Removes and returns element at `index` (or last element if not specified). Only list method that both modifies and returns a value.
- `remove(x)`: Removes first occurrence of `x` from the list (not by index); does not return value.
- `clear()`: Removes all elements, leaving an empty list.

Ordering and Copying

- `reverse()` : Reverses list in place (returns `None`, modifies the same object).
- `sort(reverse=False)`
: Sorts list in ascending (default) or descending order. Only works with homogeneous (same type) elements. Errors if mixed types (e.g., numbers and strings).

Nested Lists

- Lists can contain other lists as elements, allowing for multi-dimensional data structures.
- Access nested elements with multiple indices.

Q&A and Clarifications

- Difference between `append` and `extend` is thoroughly discussed.
- `insert` can add at any position, `append` always at end.
- Methods like `pop`, `remove`, `sort`, and `reverse` are illustrated via examples.
- Student questions highlight pitfalls, method returns (when methods return values vs. `None`), and behavior when types are mixed.
- Recap: Sets do not maintain order or allow duplicates, lists do.

Assignment and Next Topics

- Students are instructed to work on list method assignments and prepare for the next topic: tuples, shallow/deep copy, and copy method.

Notes powered by [CraftNote](#)